FIT3155: Week 4 Lab Sheet

(Questions are based on Week 3 topic, Burrows-Wheeler Transform)

Objectives: Develop a clear understanding and working implementation of Burrows-Wheeler Transform (BWT), inverting a BWT and searching a pattern using BWT as a search index.

Conceptual exercises

1. For the string $S =$ $\overset{1}{\texttt{m}}\ \overset{2}{\texttt{i}}\ \overset{3}{\texttt{s}}\ \overset{4}{\texttt{s}}\ \overset{5}{\texttt{i}}\ \overset{6}{\texttt{s}}\ \overset{7}{\texttt{s}}\ \overset{8}{\texttt{i}}\ \overset{9}{\texttt{p}}\ \overset{10}{\texttt{p}}\ \overset{11}{\texttt{i}}\ \$$, where $ is a special terminal character that is assumed to be lexicographically smaller than any other character in S.

- [x] (a) On paper, construct its sorted cyclic permutation matrix M.
- [x] (b) Identify the BWT of S from M and reason the worst-case time complexity of constructing a BWT using this approach.
- [x] (c) On paper, construct its suffix array.* Verify that the information stored in the suffix array correspond to the ordering of rows in M. Reason why this hold in general.
- [x] (d) Confirm visually that each **column** of M is a permutation of characters in S, and explain why this holds for any string.
- [x] (e) How can the BWT be computed from the information in the suffix array? Assume the you have a method to compute the suffix array of any given string. What is the time complexity of computing the BWT of a string given its suffix array?
- [x] (f) For each character $L[i]$ in the last column (L) of M, visually identify its corresponding position p in the first column (F) of M. (This is performing a visual LF-mapping on this example string.)
- [x] (g) For any $L[i] \mapsto F[p]$, verify that $L[p]$ immediately precedes $L[i]$ in S, and explain why this holds in general.

- [ ] 2. Prove the following statement involving LF-mapping: If $L[i] \mapsto F[p]$, then $p = \texttt{rank}(L[i]) + \texttt{nOccurrences}(L[i] \in L[1 \ldots i))$†

- [ ] 3. BWT of some string S is `ooolooooolwml$`. What is S?

- [ ] 4. In general, what is the worst-case time and space complexity of inverting a BWT **without** preprocessing the `rank` and `nOccurrences` functions.

*A suffix array is an array storing positions of all suffixes of a string in a sorted order.
†Range L[1...i) is exclusive of i.

5. Write pseudocode to preprocess a `rank` data structure. Characterize the worst-case time and space complexity of its construction.

6. Write pseudocode to preprocess a `nOccurrences` data structure, assuming you would use it only for inversion. Characterize the worst-case time and space complexity of its construction.

7. When BWT is inverted into its original string using preprocessed `rank` and `nOccurrences` data structures, what would be the worst-case space and time complexity of inversion (including the preprocessing steps).

8. For $\texttt{pat}[1 \ldots 3] = \texttt{iss}$:

   (a) Work out step-by-step the iterative backward search of the pattern using the BWT you have constructed for Q1 as a search index. In your working on paper, mainly identify the updates to the start-position (sp) and end-position (ep).

   (b) Understand what the updated range $L[sp \ldots ep]$ is conveying after each iteration of the backward search.

   (c) At the end of the search, how would you work out the **number** of occurrences of the pattern in the original string?

   (d) What additional information do you need to be able to identify the **loci** of occurrences of the pattern in the original string?

9. Prove the correctness of the update rules for start and end positions duing the backwards search:

   - $sp_{\text{new}} = \texttt{rank}(x) + \texttt{nOccurrences}(x \in L[1 \ldots sp))$
   - $ep_{\text{new}} = \texttt{rank}(x) + \texttt{nOccurrences}(x \in L[1 \ldots ep] - 1)$

10. Write pseudocode to preprocess a `nOccurrences` data structure for its use in pattern matching. Characterize the worst-case time and space complexity of its construction.

11. Characterize the worst-case time and space complexity to search for a pattern $\text{pat}[1 \ldots m]$ using the BWT as the search index.

Implementation exercises

(A message of caution: 'Vibe coding' with GenAI will hinder your learning of algorithms and data structures. The degree (and this unit) you have enrolled into is a specialist degree (unit). By definition it expects one to demonstrate specialist understanding of topics. This understanding comes from making a concerted effort, via thinking, implementing, debugging etc.)

- [x] 1. Write a program to construct a BWT of any input string (read from a file, whose name is passed in as an argument to your program). Note, during the initial attempt feel free to use a naïve algorithm to construct a suffix array. After next week's topic is completed, you would have learnt an asymptotically optimal (linear-time) algorithm that will allow you to compute a suffix array of very large strings.

- [x] 2. Write a program to invert any BWT provided as input. That is, the input to your program is a BWT string (again, read from a file) and your program should invert the BWT and recover the original text/string. As a part of this implementation, reason the time and space complexities of your implementation.

- [ ] 3. Write a program to find all occurrences of any input pattern within an input reference text (both inputs read from their respective files). As a part of this implementation, reason the time and space complexities of your implementation.

```
-=o0o=-
  END
-=o0o=-
```